MOUNTX SELF-HOSTING

Phase 1: Lexer

A 66-entry regex table, re-expressed by hand in a language with no regex — and proved equal to it, token for token.

product	selfhost/lexer.mx - 731 lines
oracle	lexer.py - 182 lines, 66-entry TOKEN_SPEC
curated gate	52 cases / 255 lines - PARITY OK
repo gate	59-73 files / 34,685-67,904 token lines - PARITY OK
diagnostics	6 of 6 templates reached
divergences	D3, D5 - loud by design, never silently fixed

*Every figure in this document is taken from the I-M9a brief, from the three design documents in **selfhost/**, or from a measurement taken while this PDF was generated — a line count, a git tag, or a frozen gate artifact. None of it is from memory. The commit range above is* `first-touching-commit..HEAD` *for this phase's own product file, so* `git log` *over it reproduces the phase's history.*

commit range	2e9830ea559d..c293f17
final tag	self-hosted-complete
generated	2026-10-02 against tree c293f17 (2026-10-02)

The tree named above is the tip these figures were measured against — the parent of the commit carrying this PDF. A document cannot record its own hash, so the commit that names it is `i-m9`*, the tag this work is released under.*

2 THE CONTRACT

The document that governs this phase is `selfhost/DESIGN.md`, written before a line of MountX existed. It opens with the goal — re-express `lexer.py` in MountX so that for any input the port yields the same token stream and the same diagnostic text — and it collects nine numbered decisions, each of which changes the code.

The nine decisions (DESIGN.md section 17)

#	issue	decision taken	the alternative that lost
D1	keyword lookup	module HashTable singleton, builder declared above the let	a flat if chain; measure both at M1
D2	number equality	lexeme equality (E3)	value equality, which needs a correctly-rounding parse native
D3	\uXXXX above U+00FF	out of scope, loud failure, recorded divergence	a char_from native — breaks the no-new-natives rule
D4	py_repr_char coverage	table over 0x00-0xFF, corpus restricted to it	full Unicode printability tables — disproportionate
D5	Unicode word chars	ASCII-only in the port; the divergence is loud, not silent	narrow lexer.py's \w to ASCII — a real behaviour change, needs its own approval
D6	whitespace-only short-circuit	copy it exactly	fix it in lexer.py too, which changes current behaviour
D7	how errors are compared	message strings (E2)	match process stderr — impossible, and pointless
D8	corpus output shape	lex_print streaming; MountX has no callbacks	array only, and pay section 16
D9	where the code lives	selfhost/, driver inside it, parity as its own command	libs/lexer.mx, which is in the import search path

***D9** is the one with a long-term consequence: today `fbtest.mx` is the gate, and D9 decides whether self-hosting is inside that gate or beside it. It sits beside it, in two commands of its own.*

The parity contract: E1 - E4 (DESIGN.md section 3)

	the rule	why this rule and not another
E1	token stream equality: one canonical line per token, in order, ending with an EOF line. Two implementations are equal when their emitted lines are byte-identical.	There is no way to compare a MountX token array with a Python list of Token objects from the outside, so equality is defined as a byte stream.
E2	diagnostic equality is at the message level, not the process level	lexer.py raises SyntaxError and main.py prints it verbatim because the text starts with '['. MountX cannot raise SyntaxError, so the port throws the exact message string and the harness compares that string. We do not try to match stderr.
E3	numbers are compared as lexemes, not values	lexer.py calls int()/float(), i.e. correctly-rounded parsing. The only MountX-side parser is string.tonum, which is decimal and builds fractions arithmetically, and can disagree in the last bit. So the canonical output carries the lexeme: 1.50 stays 1.50.
E4	explicitly out of scope for equality	Python repr of floats, process exit codes, printx rendering, and any diagnostic whose text comes from decode_string.

The frozen constraints

- **No host edits.** Phase 1 changes nothing in lexer.py, parser.py, main.py, interpreter.py, fbtest.mx or stdlib/. New files only, and no VERSION bump, because nothing a user can observe changed.
- **No new natives.** MountX has no chr/ord and interpreter.py has no char_code/char_from — grepped, not assumed. The workaround is a module-level literal holding code points 0x00-0xFF: indexed for code-to-char, searched with string.index_of for char-to-code.
- **No closures.** A function sees its own scope, module scope and globals, and nothing captured. Callbacks are impossible, which is why the token stream is printed rather than handed to a visitor.
- **First hit in table order wins.** Not longest-match: longest-match would silently change 1.5y and ==. The scan is total — SKIP eats every newline, MISMATCH matches every other leftover character — so the port has no skip-a-char-and-retry path.
- **A tab is one column, and \r is not a line break.** Only \n advances the line counter, so in a CRLF file the \r still occupies a column. Reproduce this; do not normalise line endings.
- **Not a performance play.** lexer.py stays the production lexer. The MountX one is a capability proof and an oracle fixture.

3 THE MEASUREMENTS

Every number on this page is a gate result, a line count taken from the tree while this PDF was generated, or a figure carried verbatim from `selfhost/DESIGN.md`.

measure	value	note
Product	`selfhost/lexer.mx, 731 lines`	a hand-rolled scanner: no regex exists in MountX
Oracle	`lexer.py, 182 lines, 66-entry TOKEN_SPEC`	one ordered alternation compiled into a single regex and driven by finditer. Table order IS priority order.
Gate - curated	`52 cases / 255 lines — PARITY OK`	22 generated case files plus 30 inline cases. The inline ones carry CR, NUL and NBSP, which a text-mode read would rewrite on both sides.
Gate - repo	`59-73 files / 34,685-67,904 token lines — PARITY OK`	every .mx and .py in the tree: the Python side walks it, writes a manifest, and the port lexes every entry. The range is the phase's own growth as files were added.
Self-lexing	`lexer.mx lexes its own source — verified`	inside the repo run: the port reads its own table literal, including its \uXXXX escapes, and matches the oracle token for token
Diagnostics	`6 of 6 templates reached`	bad float, bad number suffix, unterminated string, bad escape, \u too short, unexpected character
Port speed	`~800 appends in about 1 ms; fbtest.mx lexes in 38 s`	lex_print and render(lex(src)) take the same 38 s and print the same 6,078 lines: the array rebuild is not where the time goes
Divergences	`D3 (\uXXXX ≤ U+00FF), D5 (ASCII-only \w)`	both print only on the failure path. A gate that trips one either changes, or the decision is reopened.

The six diagnostics, and what each one proves

template	observed text	reached by
BAD_FLOAT	Invalid float '1.' - trailing dot. Use '1' or '1.0'	1..2 — the dot is followed by a dot, not a digit
BAD_NUM_SUFFIX	Invalid number '1_' - a number cannot be followed by letters. Write '1' and '_' separately	1_.5 — the quoted lexeme is the consumed run, while num/rest come from a different split. That asymmetry is visible in the output.
UNTERMINATED	Unterminated string literal (missing closing " before end of line)	"ab
bad escape	Invalid string literal - invalid escape '\q'	"a\q
\u too short	Invalid string literal - '\u' must be followed by 4 hex digits	a\u12
MISMATCH	Unexpected character '@'	a single @

*Four more messages live in decode_string and are **unreachable by construction**: a literal ending in an unescaped backslash has no closing quote, so the regex declines it first. The port implements them anyway for decode_string's own contract, and the harness is forbidden from expecting them from corpus cases. A corpus that never emits a template is either covering dead code or missing a bug — which is exactly how this count got fixed: the first draft came out one message too high, counted by counting raise sites instead of running them.*

4 DEFECTS AS LESSONS

Phase 1 found three defects, and the striking thing about them is that **none was visible in isolation**. Each produced output that was well-formed and merely wrong — which is why a curated corpus written by someone who already knows what to look for would not have found any of them.

what was caught	how	the fix	the lesson
Hex case fold. Uppercase A-F in a \uXXXX escape folded to 0 instead of 10-15, so "\uABCD" decoded to the wrong character and nothing complained.	The repo stage. A file in the tree carried a \uXXXX escape with uppercase hex; the port's decoded string differed from the oracle's in one character.	hex_value() indexes a 22-entry table (digits, then a-f, then A-F) and folds case with % 16, so 'a' at index 10 and 'A' at index 26 agree. The comment above the function says why the table's order matters.	**A silent wrong answer inside well-formed output is the worst kind of bug**, because every existing check passes. Only an independent implementation notices.
Keyword names transcribed from prose. Four token names were copied out of the design document instead of read off match.lastgroup: lenx emitted LEN rather than LENX, selfx THIS rather than THISX, nil NULL rather than NILX, throw THROW rather than THROWX.	The repo stage, on real .mx files that use the keyword aliases — exactly what a hand-written case list forgets.	The names are now derived programmatically from lexer.TOKEN_SPEC, so no list in the port can drift from the oracle's.	**Read the source, never the prose about the source.** A design document that describes a table accurately is still a human transcription of it.
The empty-source quirk. lex() short-circuits before its loop when the source is empty or all whitespace, returning one EOF at line 1, column 1 — so a source of \n\n yields EOF(1,1) where the general formula says (3,1).	A curated case: two blank lines. The port's general formula produced a correct-looking EOF at line 3; the oracle says line 1.	Copied exactly, by decision D6: an explicit early return in the port and a corpus case pinning it. The alternative was to fix lexer.py too, which would change current behaviour.	**A quirk you can see is not a bug; it is a decision you have not written down.** D6 is the written-down version.

And the two divergences that were NOT fixed

- **D3 — \uXXXX only up to U+00FF.** MountX has no chr(), and the 0x00-0xFF literal is the only character source the port has. Above it, index_of returns -1 and the port fails loudly rather than invent a character. This is not hypothetical: the first repo-wide run died on a \u20ac written into parity.mx to spell the euro case, which is why the corpus spells non-ASCII characters literally instead. A literal character costs nothing — it is a slice of the source, never a synthesis.
- **D5 — ASCII-only \w.** Python's \w is Unicode alnum, so cafe-with-e-acute is one identifier there and an error here. The fix would be to narrow lexer.py, which is a user-visible behaviour change and needs its own approval, not a side-effect of a parity run.

Both print on the failure path only. The rule the phase wrote down is the one the whole project now uses: ***do not silently fix a divergence, and do not fix one by narrowing the Python so both sides agree.*** *Passing quietly is the one forbidden outcome.*

5 THE ARTIFACTS

Two commands, two stages. The curated stage catches known bugs quickly; the repo stage is the one that finds bugs, because it walks everything.

```
# stage 1 - the curated corpus plus the inline cases
$ python3 selfhost/parity.py --check
PARITY OK: 52 cases, 255 lines

# stage 2 - every .mx and .py in the repository
$ python3 selfhost/parity.py --repo --check
PARITY OK: 141 cases, 68776 lines
EXIT=0

# 59-73 files / 34,685-67,904 token lines is the phase's own range;
# 141 is every file in the tree at report time.
```

The repo run above is the one re-executed while this PDF was generated: the Python side walked the tree, wrote the manifest `selfhost/repo_files.txt`*, and the port lexed every entry through* `selfhost/parity_repo.mx`*. The two streams were compared byte for byte.*

What each piece is

file	role
`selfhost/gen_cases.py`	writes the corpus into cases/ from one dict, not 22 files. The oracle side is generated; the port has to reach the same bytes by its own means.
`selfhost/cases/NN-*.txt`	22 corpus cases, compared byte-exactly, and checked in — a corpus you cannot diff against last week is not a corpus
`selfhost/lexer.mx`	the port
`selfhost/parity.mx`	MountX driver: the curated corpus and the inline cases
`selfhost/parity_repo.mx`	MountX driver: every .mx and .py in the repo
`selfhost/parity.py`	oracle driver: the same corpus through lexer.lex, the same encoder, and the --check that diffs the two streams
`selfhost/repo_files.txt`	the manifest, shared between runs; the lexing is not

Three traps in the harness itself, and what they teach

- **Text-mode I/O normalises CRLF to LF on both sides.** readx does it, open() does it, so a file-based case cannot express a CRLF source at all. The CRLF and lone-CR cases therefore live in INLINE in parity.py and in inline_cases() in parity.mx, spelled the same way on both sides.
- **The raw lexeme is not in the token.** lexer.py's Token keeps converted values, so 007 becomes the int 7 and the V field of the canonical line is gone. parity.py recovers it by re-scanning with lexer.TOKEN_REGEX — the same object lex() uses, not a copy of the pattern — and refuses to print anything if that scan and lex() disagree. A harness that quietly substituted a different string would hide the very bugs it exists to find.
- **Two divergence classes are named, not skipped.** D5 lists the words Python's \w accepts and the port does not; D3 lists the literals carrying \uXXXX above U+00FF.

The payoff, in the project's own words

Phase 1's real return was not the .mx file. It was that `lexer.py` — which until then had **no differential test at all**, only fbtest.mx happy paths — acquired one. That is now the trigger rule in AGENTS.md: any change to lexer.py must re-run the parity gate. The next phase inherits the shape unchanged and points it at a larger surface.

6 THE CHAIN

12 tags carry the self-hosting work, in the order they were made. The list below is read from `git for-each-ref --sort=creatordate refs/tags` at generation time, with each tag peeled to the commit it points at, so the hashes are the repository's and not this document's.

#	tag	date	commit	what it froze
1	`i-m0-baseline`	2026-10-01	`2e9830ea559d`	MountX I-M0 baseline: interp.mx value layer (10 tags, display, identity stamps, kind slot) certified by 5-gate differential. Lexer 59/46,944 + parser 49/26,501 byte-exact, fbtest 277, 67/67 oracle pins, 13-mutation control battery.
2	`i-m2-shipped`	2026-10-01	`1785ee96ce67`	I-M2 shipped: calls, control flow, value layer, natives; 8-guest exec gate green
3	`i-m2-review`	2026-10-01	`9a97b5453ccc`	MountX I-M2 review (Q1): MAX_GUEST_DEPTH 100 -> 40, because 100 was inert too
4	`i-m3-errors`	2026-10-01	`032342a92e37`	MountX I-M3: the error pass - 30-program message sweep, mk_super collision fix, D-1 search half
5	`i-m4`	2026-10-01	`469b167c4e02`	MountX I-M4: the class layer - ClassDef, New, instance/super DotAccess, Input
6	`i-m5`	2026-10-01	`4cbf5651ba1f`	MountX I-M5: the closure pass - loops proven, sweep measured, not counted
7	`i-m6-prep`	2026-10-01	`e882d58e0b97`	I-M6 prep: close I16/I17/I10, and fix a message the audit could not see
8	`i-m6`	2026-10-01	`0ac2d56830b4`	I-M6: the anchor - interp.mx runs fbtest.mx byte-for-byte
9	`i-m7`	2026-10-02	`373d319a960e`	I-M7: the layer boundary - a delegated native answers the HOST's context
10	`i-m7.1`	2026-10-02	`68528f66f9de`	I-M7.1: the crown runs - and the second defect class, an ORDER not a transcription
11	`self-hosted-complete`	2026-10-02	`68528f66f9de`	I-M7.1: the crown runs - and the second defect class, an ORDER not a transcription
12	`i-m9-docs`	2026-10-02	`40d0b5cc6a19`	I-M9: the MountX documentary, and a gate that keeps it true

Hashes are the first 12 characters of the object each tag peels to. `git rev-parse <tag>` prints the untruncated 40, so any row can be checked against the repository rather than against this document.

Where this phase sits

- **Phase 1** is frozen at the head of the chain, and `self-hosted-complete` carries the phase's claim in full.
- **Phase 1 (lexer)** and **Phase 2 (parser)** have no tags of their own. They are proven continuously by their gates, and their products — `lexer.mx` (731 lines) and `parser.mx` (1386 lines) — are unchanged across every tag in the list.
- **Phase 3 (interpreter)** owns every tag from `i-m0-baseline` onward. Of the 12 tags in the chain, 12 are phase-3 work; the newest is this documentary, released as `i-m9`.

What the chain asserts

Phase 1's claim is one sentence long, and it is a claim about bytes: **for every input, the MountX lexer emits the same token stream and the same diagnostic text as lexer.py** — proved on a curated corpus and on every file in the repository, with two divergences named out loud rather than papered over.

```
$ git --no-pager for-each-ref --sort=creatordate \
      --format='%(refname:short) %(creatordate:short) %(objectname)' \
      refs/tags
  # run at generation time; this is what the table above is
  # (add | cut -c1-12 to print hashes untruncated)
```

MountX I-M9a — documentary phase reports. Generated by `docs/make_phase_pdfs.py` on 2026-10-02 at 13:02, against tree c293f17, for tag `i-m9`.